

Modeling and Dynamics of a Quadcopter

Pineau Benjamin

August 7, 2026

Contents

1	Hypothesis	3
2	Reference Frames and Notation	3
2.1	Euler's Angles	4
3	Modeling	4
3.1	Forces	4
3.1.1	Thrush	4
3.1.2	Motor Torques	5
4	Dynamics	5
4.1	Absolute Angular Velocity	5
4.2	Mass Distribution	7
4.3	Newton-Euler's Equations	7
4.3.1	Useful formula	7
4.4	Force	7
4.5	Torques	7
5	PCB	8
5.1	Design Methodology	8
5.1.1	Layer Stackup	8
5.1.2	Component Placement	8
5.1.3	Trace Routing	8
5.2	Table of components	8
5.2.1	Component Justifications	8
5.2.2	Tables	9
6	Control Algorithm	13
6.0.1	Cascade Control	13
6.0.2	Controller Design	14
6.0.3	Acceleration Controller	14
6.0.4	Fore and Yaw to Thrust and attitude	15
6.0.5	Desired Thrust	15
6.0.6	Desired Body Z-axis	15
6.0.7	Desired Direction	15
6.0.8	Desired Body Y-axis	15

6.0.9	Desired Body X-axis	15
6.0.10	Desired Rotation Matrix	15
6.0.11	Attitude Controller	16
6.0.12	Rate Controller	16
6.0.13	Efforts vers commandes moteur	17

1 Hypothesis

- The quadcopter is assumed to be **rigid**
- The propellers are assumed to be **rigid**
- The quadcopter has 2 planes of symmetry : (x, z) et (y, z)
- The center of mass G is assumed to coincide with the geometric center.
- The moments of inertia of the motors and propellers are assumed to be negligible compared to the rest of the vehicle.
- The **Earth** is assumed to be **flat** and **stationary**.
- The **Earth** is considered to be an **inertial reference frame**.
- The interaction between the drone and the ground is neglected.
- The onboard sensors perform measurements with respect to the drone's reference frame.

2 Reference Frames and Notation

Two reference frames are defined:

- Inertial reference frame (x_i, y_i, z_i)
- Drone body-fixed reference frame (x_d, y_d, z_d)

The system is described by six parameters:

$$q = [x, y, z, \phi, \theta, \psi] = [\xi, \eta]$$

where

$$\xi = [x, y, z]$$

represents the position coordinates, and

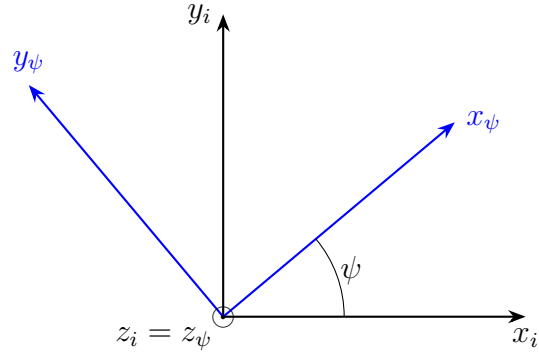
$$\eta = [\phi, \theta, \psi]$$

represents the orientation parameters.

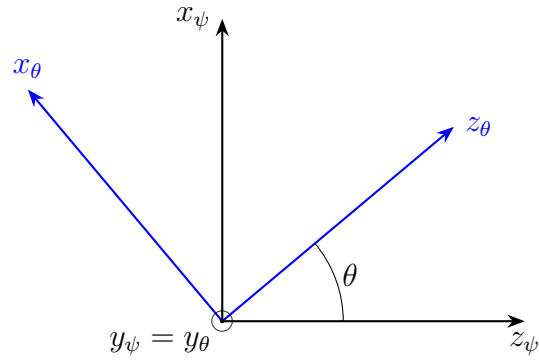
The following Euler angles are defined:

- ϕ : roll angle
- θ : pitch angle
- ψ : yaw angle

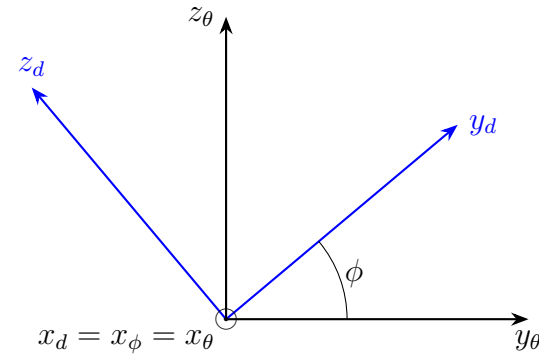
2.1 Euler's Angles



(a) Rotation ψ



(b) Rotation θ



(c) Rotation ϕ

Figure 1: Euler's angles

3 Modeling

3.1 Forces

3.1.1 Thrush

The thrust generated by each motor is given by

$$T_i = K_T \omega_i^2$$

where K_T is the thrust coefficient and ω_i is the angular speed of the i -th motor. Therefore,

$$T = T_1 + T_2 + T_3 + T_4 = K_T(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2).$$

3.1.2 Motor Torques

The torques generated by the motors are given by

$$\tau_x = lk(\omega_4^2 - \omega_2^2)$$

$$\tau_y = lk(\omega_3^2 - \omega_1^2)$$

$$\tau_z = b(\omega_2^2 + \omega_4^2 - \omega_1^2 - \omega_3^2)$$

where l is the distance between the center of mass and each motor, and k is the lift coefficient.

The resulting torque vector is therefore

$$\vec{\tau} = \tau_x \vec{x}_d + \tau_y \vec{y}_d + \tau_z \vec{z}_d.$$

4 Dynamics

4.1 Absolute Angular Velocity

Let

$$\vec{\Omega} = \dot{\phi} \vec{x}_d + \dot{\theta} \vec{y}_d + \dot{\psi} \vec{z}_\psi$$

denote the absolute angular velocity of the drone with respect to the inertial reference frame.

However, this expression is defined using intermediate axes. Since the drone's inertia tensor is constant only in the body-fixed frame, it is desirable to express the angular velocity in the drone frame.

To this end, we define the following frame transformation matrices (note that these differ from rotation matrices, as the adopted convention is the inverse):

$${}^i R_\psi = \begin{pmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

and :

$$\begin{pmatrix} x_\psi \\ y_\psi \\ z_\psi \end{pmatrix} = \begin{pmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_i \\ y_i \\ z_i \end{pmatrix}$$

$${}^\psi R_\theta = \begin{pmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{pmatrix}$$

and :

$$\begin{pmatrix} x_\theta \\ y_\theta \\ z_\theta \end{pmatrix} = \begin{pmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{pmatrix} \begin{pmatrix} x_\psi \\ y_\psi \\ z_\psi \end{pmatrix}$$

$${}^\theta R_\phi = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & \sin \phi \\ 0 & -\sin \phi & \cos \phi \end{pmatrix}$$

and :

$$\begin{pmatrix} x_\phi \\ y_\phi \\ z_\phi \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & \sin \phi \\ 0 & -\sin \phi & \cos \phi \end{pmatrix} \begin{pmatrix} x_\theta \\ y_\theta \\ z_\theta \end{pmatrix}$$

Thus :

$$c_\phi = \cos \phi, \quad s_\phi = \sin \phi, \quad c_\theta = \cos \theta, \quad s_\theta = \sin \theta, \quad c_\psi = \cos \psi, \quad s_\psi = \sin \psi.$$

$${}^i R_d = \begin{pmatrix} c_\theta c_\psi & c_\theta s_\psi & -s_\theta \\ -c_\phi s_\psi + s_\phi s_\theta c_\psi & c_\phi c_\psi + s_\phi s_\theta s_\psi & s_\phi c_\theta \\ s_\phi s_\psi + c_\phi s_\theta c_\psi & -s_\phi c_\psi + c_\phi s_\theta s_\psi & c_\phi c_\theta \end{pmatrix}$$

We define

$$\vec{\Omega}_{R_d} = p\vec{x}_d + q\vec{y}_d + r\vec{z}_d,$$

the absolute angular velocity vector expressed in the body-fixed reference frame R_d , which is obtained using the following frame transformation matrices:

$$\vec{\Omega}_{R_d} = \dot{\psi} {}^\theta R_\phi {}^\psi R_\theta \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}_{(x_\psi, \vec{y}_\psi, z_\psi)} + \dot{\theta} {}^\theta R_\phi \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}_{(x_\theta, \vec{y}_\theta, z_\theta)} + \dot{\phi} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}_{(x_\phi, \vec{y}_\phi, z_\phi)}$$

$$\begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{pmatrix} \dot{\phi} - \dot{\psi} S_\theta \\ \dot{\theta} C_\phi + \dot{\psi} C_\theta S_\phi \\ -\dot{\theta} S_\phi + \dot{\psi} C_\phi C_\theta \end{pmatrix}$$

We derive the mapping matrix W^{-1} (note that this is not a reference frame transformation matrix, since components expressed in different frames have been combined. Therefore, it no longer represents a single physical vector attached to a moving body), which relates $\vec{\Omega}_{R_d}$ and $\vec{\Omega}$:

$$\vec{\Omega}_{R_d} = W^{-1} \vec{\Omega}.$$

$$W^{-1} = \begin{pmatrix} 1 & 0 & S_\theta \\ 0 & C_\phi & C_\theta S_\phi \\ 0 & -S_\phi & C_\phi C_\theta \end{pmatrix}$$

4.2 Mass Distribution

Using the 2 planes of symmetry, the inertia matrix form is : $I = \begin{pmatrix} I_x & 0 & 0 \\ 0 & I_y & 0 \\ 0 & 0 & I_z \end{pmatrix}$

To identify the moments of inertia, the *bifilar pendulum* experience is conducted.

4.3 Newton-Euler's Equations

4.3.1 Useful formula

The time derivative of a vector $\vec{i}$ in a frame B relative to a frame A is given by the following formula:

$$\frac{{}^B d\vec{i}}{dt} = \frac{{}^A d\vec{i}}{dt} + \vec{\Omega}_A \times \vec{i}$$

where $\vec{\Omega}_A$ is the angular velocity relative to B expressed in the frame A.

4.4 Force

First, we define the vector $\vec{v}_d = u\vec{x}_d + v\vec{y}_d + w\vec{z}_d$ as the velocity of the drone relative to the inertial frame expressed in R_d (as measurements will be carried out in this frame).

Thus:

$$\begin{aligned} \sum \vec{F} &= m \frac{{}^{R_d} d}{dt} \vec{v}_d = m \left(\frac{{}^{R_d} d}{dt} \vec{v}_d + \vec{\Omega}_{R_d} \times \vec{v}_d \right) \\ \sum \vec{F} &= m \begin{pmatrix} \dot{u} + w(\dot{\theta}C_\phi + \dot{\psi}C_\theta S_\phi) - v(\dot{\psi}C_\theta C_\phi - \dot{\theta}S_\phi) \\ \dot{v} - w(\dot{\phi} - S_\theta\dot{\psi}) + u(-\dot{\theta}S_\phi + \dot{\psi}C_\theta C_\phi) \\ \dot{w} + v(\dot{\phi} - \dot{\psi}S_\theta) - u(\dot{\theta}C_\phi + \dot{\psi}C_\theta S_\phi) \end{pmatrix}_{x_d, \vec{y}_d, z_d} \\ &= m \begin{pmatrix} mgS_\theta \\ -mgC_\theta S_\phi \\ T - mgC_\theta C_\phi \end{pmatrix}_{x_d, \vec{y}_d, z_d} = m \begin{pmatrix} \dot{u} + wq - vr \\ \dot{v} - wp + ur \\ \dot{w} + wp - uq \end{pmatrix}_{x_d, \vec{y}_d, z_d} \end{aligned}$$

Expressed in the inertial frame :

$$\vec{F} = \begin{pmatrix} T(\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \\ T(\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \\ T \cos \phi \cos \theta - mg \end{pmatrix}_{x_i, \vec{y}_i, z_i}$$

4.5 Torques

For the torques, we use the same method and we get :

$$\begin{pmatrix} \tau_x \\ \tau_y \\ \tau_z \end{pmatrix}_{x_d, \vec{y}_d, z_d} = \begin{pmatrix} I_x \dot{p} + qr(I_z - I_y) \\ I_y \dot{q} + pr(I_x - I_z) \\ I_z \dot{r} + pq(I_y - I_x) \end{pmatrix}_{x_d, \vec{y}_d, z_d}$$

5 PCB

5.1 Design Methodology

The methodology can be divided into two parts: component placement and trace routing.

5.1.1 Layer Stackup

A 4-layer stackup was used: a signal layer (top), a ground plane (mid 1), a power plane (mid 2), and a second ground plane (bottom) to handle trace crossovers if needed.

Rationale: 1) Standardizing the plane layers ensures consistent shielding. 2) Routing signals on the top layer prevents capacitive coupling that could create parasitic fields. 3) A 4-layer board was chosen because PCB manufacturers standardly offer 2, 4, or 8 layers, and the design required at least 3.

5.1.2 Component Placement

For this flight controller, the design rationale was as follows: since the drone frame was not yet finalized during the design phase, the IMU was placed at the board's center to ensure it aligned with the geometric center, keeping future mounting options open. Next, the LDO (Low Dropout Regulator, chosen over a switching regulator for its lower noise profile) was placed as far as possible from sensitive components to minimize electromagnetic interference (EMI).

The remaining components were then arranged to optimize overall board space utilization.

In hindsight, if I were to redesign this board, I would place the STM (microcontroller)—or the component with the highest pin count—at the center and route around it. Placing the STM on the edge made routing significantly more challenging than simply shifting the board's mounting position would have been.

5.1.3 Trace Routing

For trace routing, trace crossovers were avoided, and 90-degree angles were strictly excluded. Low-current, low-frequency signals were routed with a trace clearance of 0.15 mm to 0.25 mm (depending on available space). For high-frequency signals (clock lines) or high-current traces (power lines), clearance was increased to 0.5 mm or the maximum achievable, as the main goal was to maximize isolation for these lines.

5.2 Table of components

5.2.1 Component Justifications

- **STM32H7:** A high-performance microcontroller chosen to avoid future processing bottlenecks, as I plan to implement autonomous navigation features. It relies on an external 8,MHz crystal for system clocking.
- **BMP280:** A barometric pressure sensor used for altitude estimation, which is essential for sensor fusion (especially during autonomous descents or advanced physical modeling).

- **ICM-42688-P:** A high-performance, low-noise IMU. Alongside the microcontroller, I consider this one of the two core components worth investing extra budget into.
- **W25Q128JV:** A Flash memory chip acting as a black box to record flight data for post-flight analysis. Data can be accessed directly via the USB-C port.
- **SWD:** Serial Wire Debug port. Optional in theory, but practically essential for easily flashing the STM, debugging code, and monitoring variables in real time—a must-have for development.
- **UART Port:** An onboard UART port for sending and receiving control commands. Currently, I use an ESP32 connected via Bluetooth to a PS4 controller; since I am focusing on stabilized flight (non-acro mode), long-range capability is not required. Using an ESP32 is much more cost-effective than buying a dedicated transceiver module, while offering multi-purpose utility.

5.2.2 Tables

Below is the list of connections. In some cases, I could have favored SPI over I2C; however, to gain hands-on experience with different protocols, I chose to diversify them as much as possible, provided it did not compromise performance.

BMP280		
Nom du pin	Broche STM32	Fonction
VDD	3V3	Power
GND	GND	Ground
GND	GND	Ground
VDDIO	3V3	Power
SD0	GND	I2C Adress
SDI	PB11	I2C2 SDA
CSB	3.3V	VDDIO
SCK	PB10	I2C2 SCL

Table 1: Connexions du BMP280

ICM-42688-P		
Nom du pin	Broche STM32	Fonction
VDD	3V3	Power
VDDIO	3V3	Power
RESV(7)	GND	Ground
RESV(2,3,10)	Not Connected	ot Connected
AP CS	PA10	GPIO
AP SDI	PB15	SPI2 MOSI
AP SCLK	PA9	SPI2 SCK
AP SDO	PB14	SPI2 MISO
INT1	PB13	EVENTOUT GPIOexti
INT2	GND	Ground

Table 2: Connexions du ICM-42688-P

W25Q128JV		
Nom du pin	Broche STM32	Fonction
VCC	3V3	Power
GND	GND	Ground
CS	PC10	GPIO
WP	3V3	Open Write Protection
HOLD or RESET	3V3	Active Low
CLK	PB3	SPI3 SCK
DI	PC12	SPI3 MOSI
DO	PC11	SPI3 MISO

Table 3: Connexions du W25Q128JV

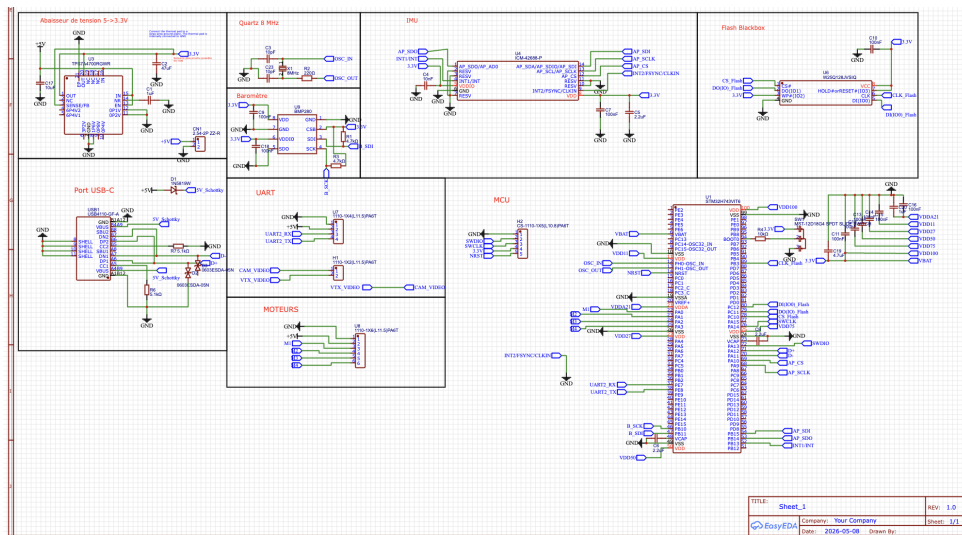


Figure 2: Diagram of the Flight Controller

Sigaux Moteurs		
Nom du pin	Broche STM32	Fonction
M1	PA0	TIM5 CH1
M2	PA1	TIM5 CH2
M3	PA2	TIM5 CH3
M4	PA3	TIM5 CH4

Table 4: Connexions du Sigaux Moteurs

Signaux de Commande UART		
Nom du pin	Broche STM32	Fonction
Rx	PE7	UART7 Rx
Tx	PE8	UART7 Tx

Table 5: Connexions du Signaux de Commande UART

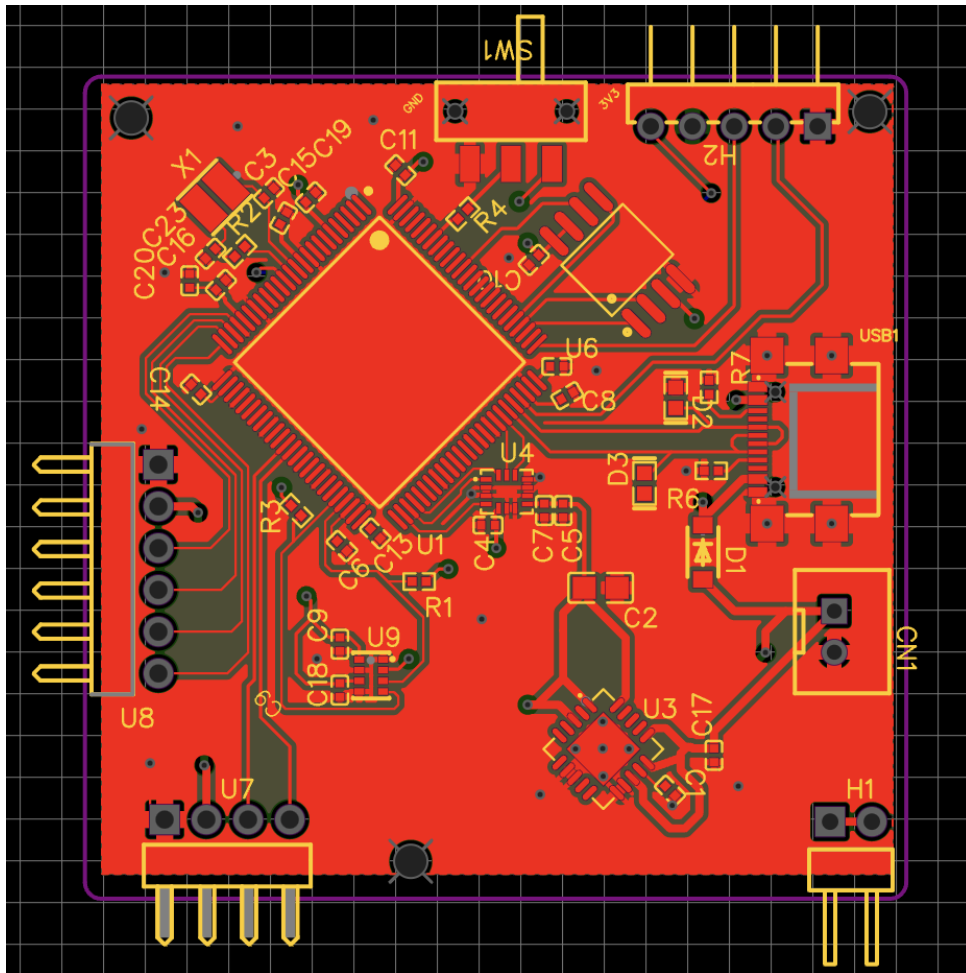


Figure 3: Upper layer of the PCB (signal layer)

USB-C		
Nom du pin	Broche STM32	Fonction
D+	PA12	OTG FS DP
D-	PA11	OTG FS DM

Table 6: Connexions du USB-C

Serial Wire Debug		
Nom du pin	Broche STM32	Fonction
3V3	3V3	Power
GND	GND	Ground
SWCLK	PA14	Clock
SWDIO	PA13	Data
NRST	14	Reset

Table 7: Connexions du Serial Wire Debug

6 Control Algorithm

We now move on to the controller. This is a particularly interesting part with no single right answer, so I will break down the underlying reasoning. As mentioned earlier, my current goal is to achieve stabilized flight—that is, a low-dynamics flight mode where pilot inputs are higher-level compared to acro mode, where the pilot performs most of the manual corrections: the left joystick controls horizontal acceleration (or thrust), the right joystick controls the yaw rate (angular velocity about the vertical axis), and two buttons control vertical acceleration. I will now rewrite the dynamic equations to make them more tractable:

$$m\vec{a} = \vec{F} + m\vec{g}$$

$$m\vec{a} = T^d R_i \vec{z}_d + m\vec{g} \quad (1)$$

$$\vec{\tau} = J\dot{\vec{\Omega}}_{R_d} - \vec{\Omega}_{R_d} \times J\vec{\Omega}_{R_d} \quad (2)$$

(Aerodynamic effects other than thrust are treated as disturbances and therefore do not appear in 1)

Let us introduce the concept of **cascade control**:

6.0.1 Cascade Control

. This is a relatively advanced yet highly useful technique, making it worth delving into in more detail. In a standard control problem, one stabilizes a single transfer function, $T(s)$. This function is generally composed of simpler functions $H_i(s)$ (first- or second-order); we can therefore write:

$$T(s) = \prod_{i=1}^n H_i(s) \quad (3)$$

However, under certain assumptions, these functions can be decoupled and stabilized independently.

Consequently, we only need to stabilize first- or second-order systems with few inputs and outputs, rather than dealing with a large, heavily coupled system.

To achieve this, the subsystems must operate at significantly different speeds—typically with at least a factor of 4 to 5 between their bandwidths—arranged in a hierarchical order. The tuning starts from the fastest loop (inner loop), moves to the second, and continues up to the slowest loop (outer loop).

Why does this work? Consider the drone example in 4. The controllers operate as closed loops, where the Acceleration Controller represents the slowest loop and the Rate Controller the fastest.

The Acceleration Controller generates a desired attitude. This attitude acts as the reference for the Attitude Controller, which drives the actual attitude toward this target. The Attitude Controller then computes an angular velocity command for the Rate Controller, which regulates the drone’s angular rates.

Assuming the inner loop operates much faster than the outer loop, the reference input is tracked almost instantaneously from the perspective of the outer loop.

Take the rate controller as an example: it receives an angular velocity command ω_d . By the time the attitude controller executes its next iteration, the rate controller will have already stabilized the system such that $\omega = \omega_d$. (Mathematically, this means that from the outer loop's perspective, the inner loop exhibits an apparent gain close to 1, as its output follows the reference nearly instantaneously.)

Thus, from the viewpoint of the outer loop, the inner loop is virtually transparent: it tracks its reference fast enough for its dynamics to be neglected when designing the outer controller.

6.0.2 Controller Design

We now move on to the design of the controller. To determine its architecture, I start from the actuators and work back to the control input. The actuators in this case are the motors, which can be abstracted to the rotational speed of the propellers.

These speeds allow control over both the torques and the total thrust of the drone. Let us address the torques first. Using 2, we trace back to the drone's angular velocities. Integrating these angular velocities yields the drone's attitude.

This attitude, combined with the total thrust, governs the drone's acceleration (1), which represents the reference input provided by the pilot.

This reasoning leads to the overall controller architecture shown in 4.

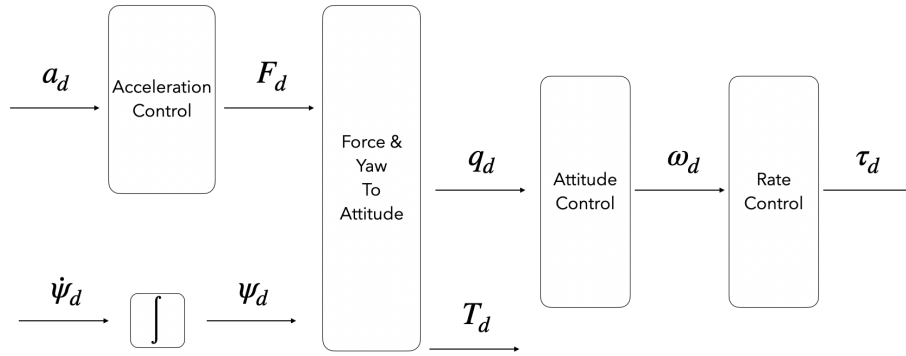


Figure 4: Global architecture of the controller

Now, let's move on onto the specific controllers :

6.0.3 Acceleration Controller

Using the equation 1, and adding a PI compensation for the perturbations :

$$\vec{F}_d = m \left(\vec{a}_d + K_{p,a} \vec{e}_a + K_{i,a} \int \vec{e}_a dt \right) - m \vec{g} \quad (4)$$

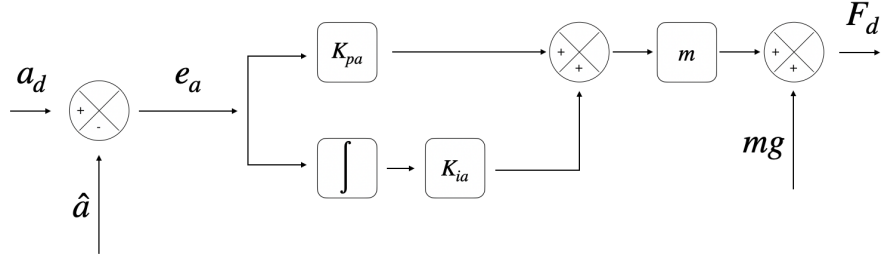


Figure 5: Architecture of the Acceleration Controller

6.0.4 Fore and Yaw to Thrust and attitude

This section is not a controller but the mathematical operations to turn desired force and yaw into thrush and attitude :

6.0.5 Desired Thrust

$$T_d = \|F_d\|$$

The steps to build the desired attitude :

6.0.6 Desired Body Z-axis

$$b_{3,d} = \frac{F_d}{\|F_d\|}$$

6.0.7 Desired Direction

$$b_{1,c} = \begin{bmatrix} \cos \psi_d \\ \sin \psi_d \\ 0 \end{bmatrix}$$

6.0.8 Desired Body Y-axis

$$b_{2,d} = \frac{b_{3,d} \times b_{1,c}}{\|b_{3,d} \times b_{1,c}\|}$$

6.0.9 Desired Body X-axis

$$b_{1,d} = b_{2,d} \times b_{3,d}$$

6.0.10 Desired Rotation Matrix

$$R_d = [b_{1,d} \quad b_{2,d} \quad b_{3,d}]$$

This matrix is then converted into quaternions q_d .

6.0.11 Attitude Controller

The quaternion error :

$$q_e = q_d^{-1} \otimes q$$

The attitude error (which we can use in a controller contrary to the quaternion error) :

$$e_R = 2 \operatorname{sgn}(q_{e,w}) q_{e,v}$$

Thus, the controller (P because there are no external perturbations here).

$$\omega_d = K_R e_R \quad (5)$$

If using a feedforward :

$$\omega_d = K_R e_R + \omega_{ff} \quad (6)$$

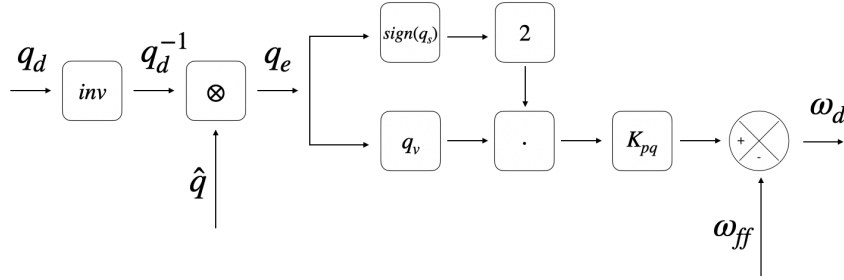


Figure 6: Architecture of the Attitude Controller

6.0.12 Rate Controller

Angular velocity error :

$$e_\omega = \omega_d - \omega$$

Thus the rate controller :

$$\tau = K_P e_\omega + K_I \int e_\omega dt + J \dot{\omega}_d \quad (7)$$

Here again, an integral term is included to compensate for disturbances that can slow down (or, more rarely, accelerate) the propeller, such as aerodynamic effects. It also accounts for battery discharge, which gradually reduces the supplied power.

Finally, it serves to compensate for model uncertainties.

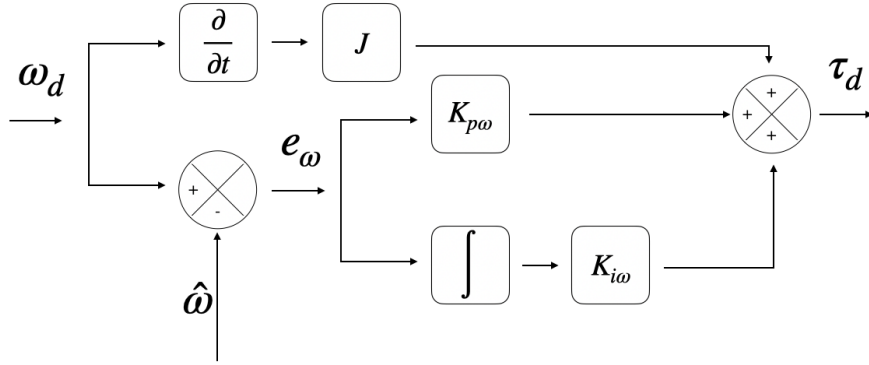


Figure 7: Architecture of the Rate Controller

6.0.13 Efforts vers commandes moteur

Finally, all that's left is to solve this system :

$$\begin{cases} \tau_x = l * k * (\omega_4^2 - \omega_2^2) \\ \tau_y = l * k * (\omega_3^2 - \omega_1^2) \\ \tau_z = b * (\omega_2^2 + \omega_4^2 - \omega_1^2 - \omega_3^2) \\ Td = K_T(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2) \end{cases}$$

which, using ω^2 as input can be turn into a linear model. The coefficient matrix is then 4x4 and can be easily inverted :

$$\begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{2lk} & -\frac{1}{4b} & \frac{1}{4K_T} \\ -\frac{1}{2lk} & 0 & \frac{1}{4b} & \frac{1}{4K_T} \\ 0 & -\frac{1}{2lk} & -\frac{1}{4b} & \frac{1}{4K_T} \\ \frac{1}{2lk} & 0 & \frac{1}{4b} & \frac{1}{4K_T} \end{bmatrix} \begin{bmatrix} \tau_1 \\ \tau_2 \\ \tau_3 \\ T_d \end{bmatrix} \quad (8)$$